DineEase — Technical Viva Guide

Full-Stack Restaurant & Café Management System (React 18, Node.js, Express, MongoDB)

Architecture: MERN Stack | Security: JWT Access + Refresh Cookies | Currency: INR (₹)

1. Project Abstract & Core Objectives

DineEase is a modern full-stack web application built to digitize and automate the dining workflow for customers, restaurant staff, and administrative management. It solves real-world rush-hour congestion, eliminates customer wait times, automates pre-ordering, and enforces secure role-based table reservations.

Key Real-World Benefits (Why This Project is Helpful):

- Zero Table Wait Time: Customers book tables & pre-order food so dishes are served fresh upon arrival.
- Kitchen Efficiency: Chefs receive pre-orders in advance, reducing peak rush hour bottlenecks.
- Multi-Restaurant Outlet Support: Manages multiple Indian restaurant brands under one portal.
- Automated Staff Desk: Staff track incoming reservations & update table seating status in real-time.

2. Technology Stack & Technical Justification ("Why We Used It")

React.js (v18) (Frontend Framework): Component-driven, Virtual DOM for sub-second UI updates without page reloads.

Vite (Build Tool / Dev Server): Sub-200ms HMR build tool, significantly faster than legacy Create React App (Webpack).

Tailwind CSS (Styling Framework): Utility-first styling with dark/light themes, ambient glassmorphism, & responsive cards.

Axios (HTTP Client): Automatic 401 JWT refresh token interceptor retries failed calls transparently.

Node.js & Express (Backend Runtime & API): Non-blocking I/O event loop capable of high concurrency; clean REST route handlers.

MongoDB & Mongoose (Database & ODM): Flexible NoSQL document model perfect for nested arrays like pre-ordered food items.

JWT (jsonwebtoken) (Authentication): Stateless auth using 15m Access Tokens in memory & 7d Refresh Tokens in HttpOnly cookies.

bcryptjs & Helmet (Security & Hashing): 10-round salted password hashing & HTTP security header enforcement against DDoS.

3. Security & System Architecture Workflows

A. JWT Token Rotation Security Architecture:

1. User Login: Server issues a 15-minute Access Token in memory and sets a 7-day Refresh Token as an HttpOnly, SameSite=Lax cookie.
2. Why HttpOnly Cookie? JavaScript cannot access HttpOnly cookies, protecting the refresh token against Cross-Site Scripting (XSS) attacks.
3. Axios Interceptor: If an access token expires, Axios intercepts the 401 Unauthorized response, silently hits /api/auth/refresh, receives a new access token, and retries the request seamlessly.

B. Multi-Step Table Reservation & Pre-Order Flow:

Step 1 (Outlet Selection): Select from Bukhara Royal Grill, Punjab Dhaba, Dakshin Tiffin Room, or Hyderabadi Shahi Kitchen.
Step 2 (Table & Time): Pick party size (1-20), reservation date, time slot, and seating area (Window, Outdoor, Booth).
Step 3 (Food Pre-Order): Add dishes to pre-order cart with live ₹ INR price calculation.
Step 4 (Confirmation): Receives booking reference & saves booking to database & localStorage tracking.

4. Top 12 Expected Viva / Interview Questions & Model Answers

Q1: What is the architectural model of your DineEase project?
DineEase follows a Client-Server RESTful Architecture (MERN Stack). The React SPA frontend communicates asynchronously with the Node.js/Express API backend over JSON HTTP requests.

Q2: Why did you choose React instead of traditional HTML/JS or PHP?
React utilizes a Virtual DOM and component state. It dynamically re-renders UI components (like live pre-order bill calculation & availability toggles) instantaneously without refreshing the browser page.

Q3: Explain how JWT Access Tokens and Refresh Tokens work in your application.
Access Tokens are short-lived (15 min) and kept in memory for Bearer Authorization headers. Refresh Tokens are long-lived (7 days) and stored in secure HttpOnly cookies. When an access token expires, /api/auth/refresh issues a new one automatically.

Q4: Why store the Refresh Token in an HttpOnly cookie instead of localStorage?
localStorage is readable by client-side JavaScript, leaving tokens vulnerable to XSS attacks. HttpOnly cookies cannot be accessed via JavaScript, providing robust protection against token theft.

Q5: How does your frontend automatically handle expired Access Tokens?
We configured an Axios Response Interceptor in axiosInstance.js. On a 401 Unauthorized status, it automatically calls /api/auth/refresh, updates memory token, and retries the original API call transparently.

Q6: Why did you choose MongoDB over SQL databases (MySQL/PostgreSQL)?
MongoDB is a NoSQL document database using JSON-like BSON format. It handles nested array structures like preOrderedItems (name, price, quantity) natively without complex relational SQL joins.

Q7: How is Role-Based Access Control (RBAC) enforced in your system?
On frontend, <ProtectedRoute allowedRoles={["admin", "staff"]}> guards pages. On backend, Express authorize("admin", "staff") middleware verifies JWT role before permitting administrative CRUD operations.

Q8: How does the food pre-ordering feature work during table reservation?
During Step 3 of the reservation wizard, users select dishes from their chosen outlet. Items, quantities, and total bill in ₹ Rupees are saved directly with the reservation document in the database.

Q9: What security measures are implemented on your Express server?
1) Helmet middleware for HTTP security headers. 2) Express Rate Limiting against brute-force DDoS. 3) Bcryptjs 10-round salted password hashing. 4) CORS locking to frontend domain.

Q10: How do you handle non-logged-in guest reservations?
All prices use Indian Rupees format ₹{price.toLocaleString("en-IN")}. For non-logged-in guests, created reservation IDs are stored in localStorage, allowing MyBookingsPage to query and display their exact booking.

Q11: How does the Admin/Staff Management Desk work?
Staff/Admin can toggle menu availability in 1 click, add/edit dishes, view incoming reservations, and update booking status (Pending -> Confirmed -> Seated -> Completed -> Cancelled).

Q12: How does the Dual Payment Gateway (Cash vs QR Code Scan) work?
For pre-ordered meals, users can choose between Instant UPI QR Code Scan (supports GPay, PhonePe, Paytm with dynamic transaction IDs) or Cash Payment (Cash on Delivery or Pay at Counter after dining). Both modes are tracked in MongoDB.

Q13: How does the system handle different modes of delivery and service?
DineEase supports 3 service modes: Dine-In Table (with seating preference: Window, Patio, Booth), Takeaway Pickup, and Doorstep Delivery (with street address validation). All are saved in the reservation document.

Q14: How does the Menu Catalog organize dishes by restaurant and meal time?
Using URL search parameters (/menu?restaurant=...), users can explore distinct restaurant outlets and filter dishes by meal timing: Breakfast, Starters, Lunch, Snacks, Dinner, and Sweets, fetched live from MongoDB.

Q15: How would you scale this application for production deployment?
Deploy React frontend to Vercel CDN, host Express backend on AWS EC2, connect to a MongoDB Atlas cluster, and integrate Redis for menu query caching & session blacklisting.